BITS Pilani
K K Birla Goa Campus

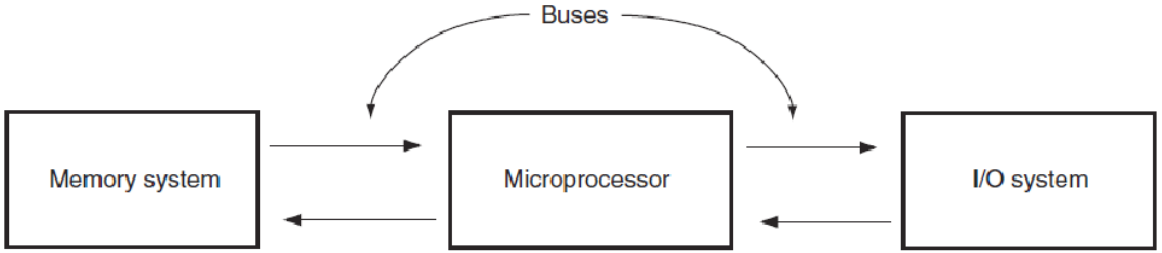
ARCHITECTURE

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS

innovate achieve lead

The Microprocessor-based Computer System



Dynamic RAM (DRAM)
Static RAM (SRAM)
Cache
Read-only (ROM)
Flash memory
EEPROM
SDRAM
RAMBUS
DDR DRAM

8086
8088
80186
80188
80286
80386
80486
Pentium
Pentium Pro
Pentium II
Pentium III
Pentium 4
Core2

Printer
Serial communications
Floppy disk drive
Hard disk drive
Mouse
CD-ROM drive
Plotter
Keyboard
Monitor
Tape backup
Scanner
DVD

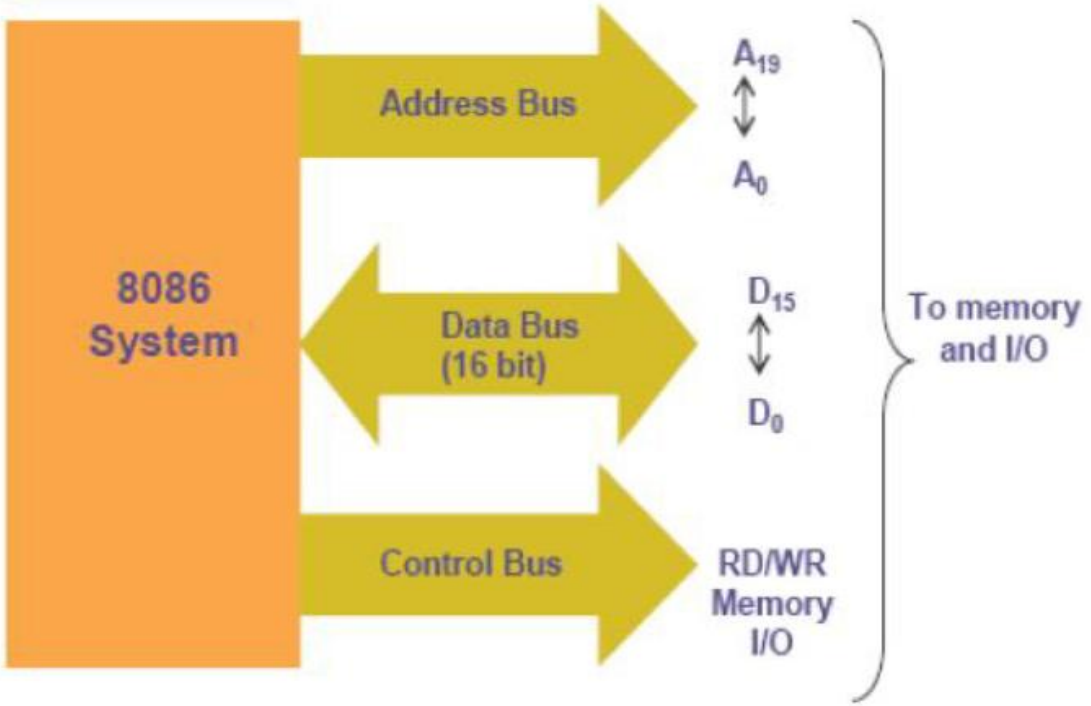
A bus is a common group of wires that interconnect components in a computer system.

Buses

Address Bus provides a memory address to the system memory and I/O address to the system I/O devices

Data Bus transfers data between the microprocessor and the memory and I/O attached to the system

Control Bus provides control signals that cause the memory or I/O to perform a read or write operation

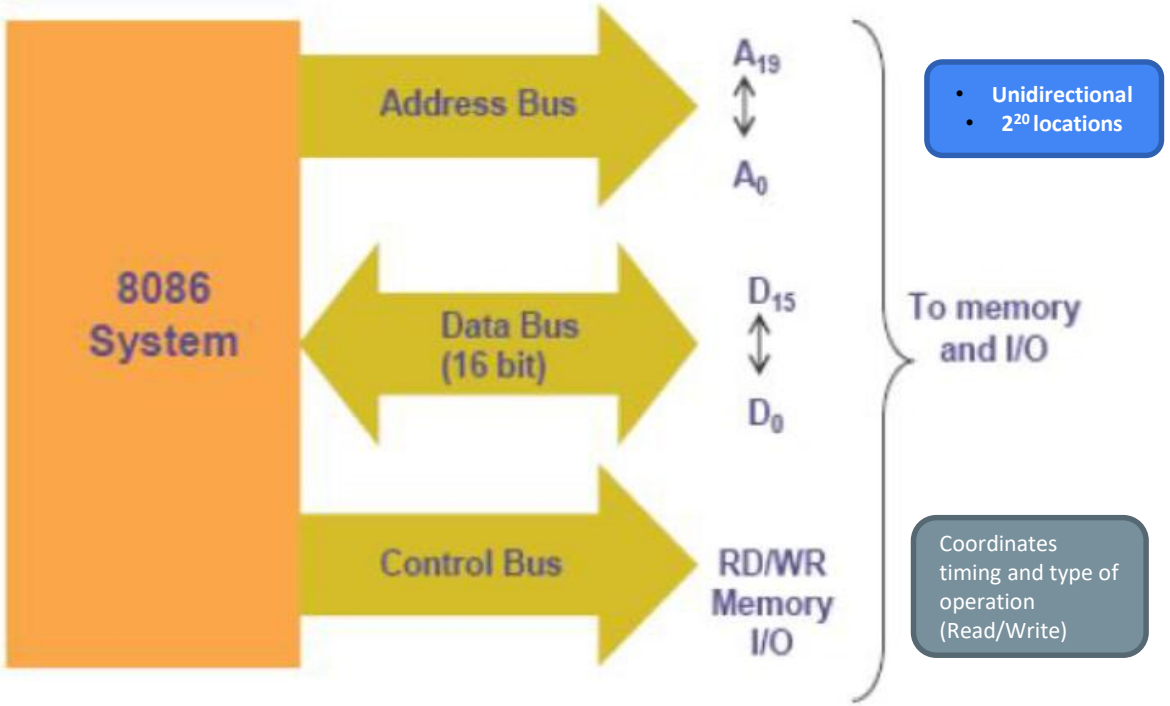


Buses

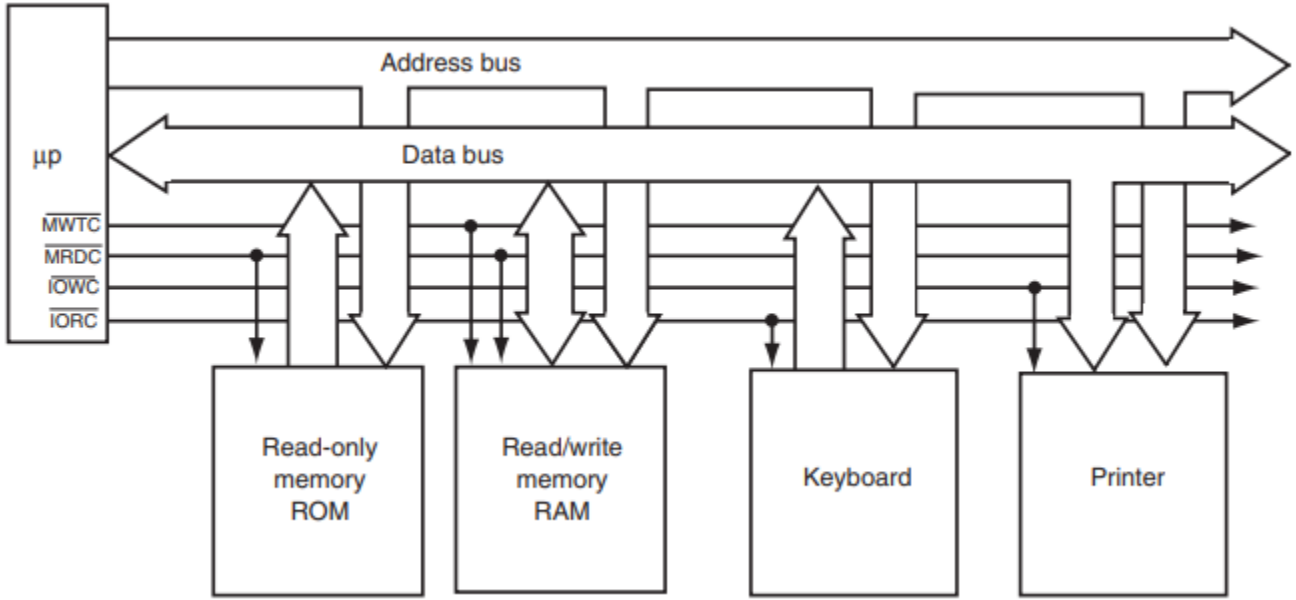
Address Bus provides a memory address to the system memory and I/O address to the system I/O devices

Data Bus transfers data between the microprocessor and the memory and I/O attached to the system

Control Bus provides control signals that cause the memory or I/O to perform a read or write operation



BUSES



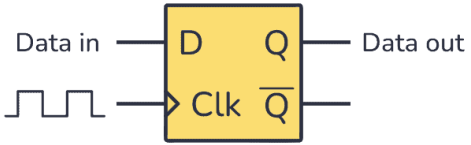
Basic control signals: **control signal is active-low**

MRDC (memory read control), MWTC (memory write control),

IORC (I/O read control), and IOWC (I/O write control).

Register

A **register** is a small, high-speed storage location located directly inside the CPU.

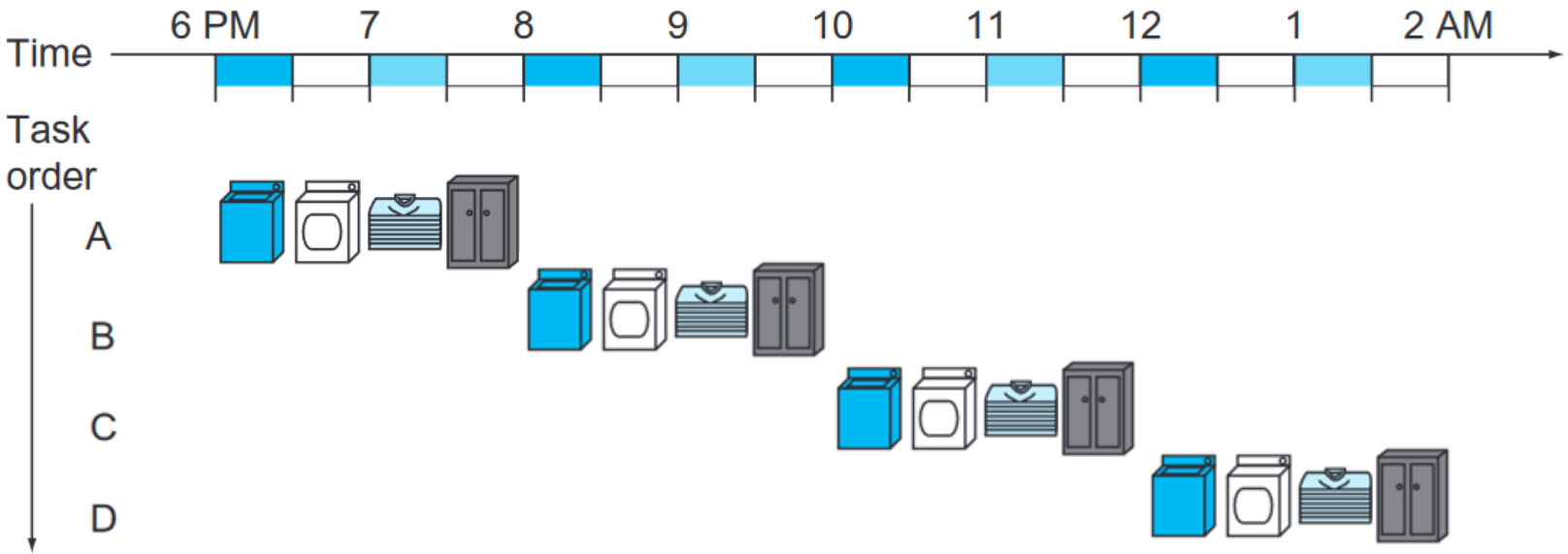


0	1	0	1	0	1	0	1

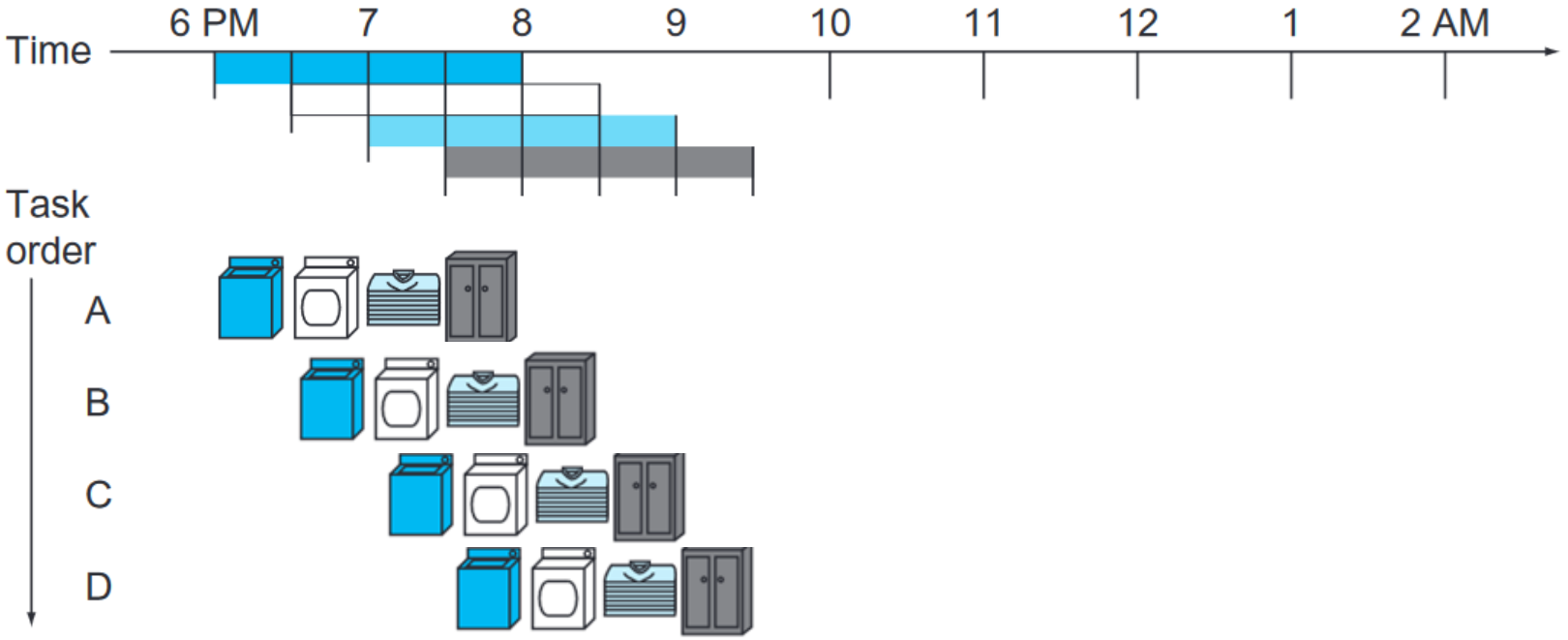
- A register is essentially a collection of **D Flip-Flops** (DFFs) arranged in parallel and synchronized by a single clock signal.
- Since one D Flip-Flop can store exactly **1 bit** of data, an n-bit register is built by connecting n flip-flops together.

Pipelining

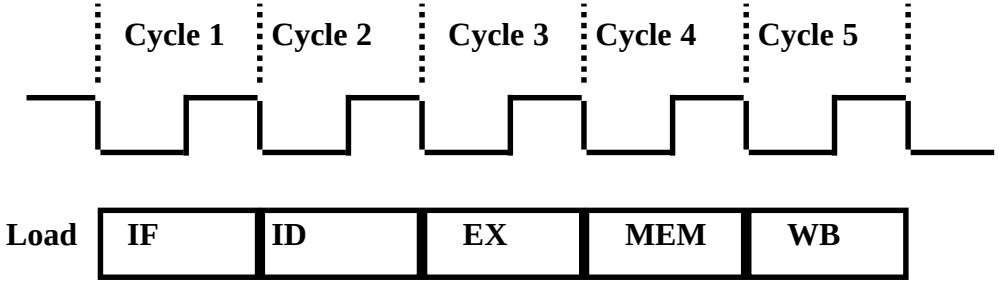

Washing
Drying
Folding
Putting Away


Time
6 PM
7
8
9
10
11
12
1
2 AM
Task order
A
B
C
D

Pipelining approach to laundry



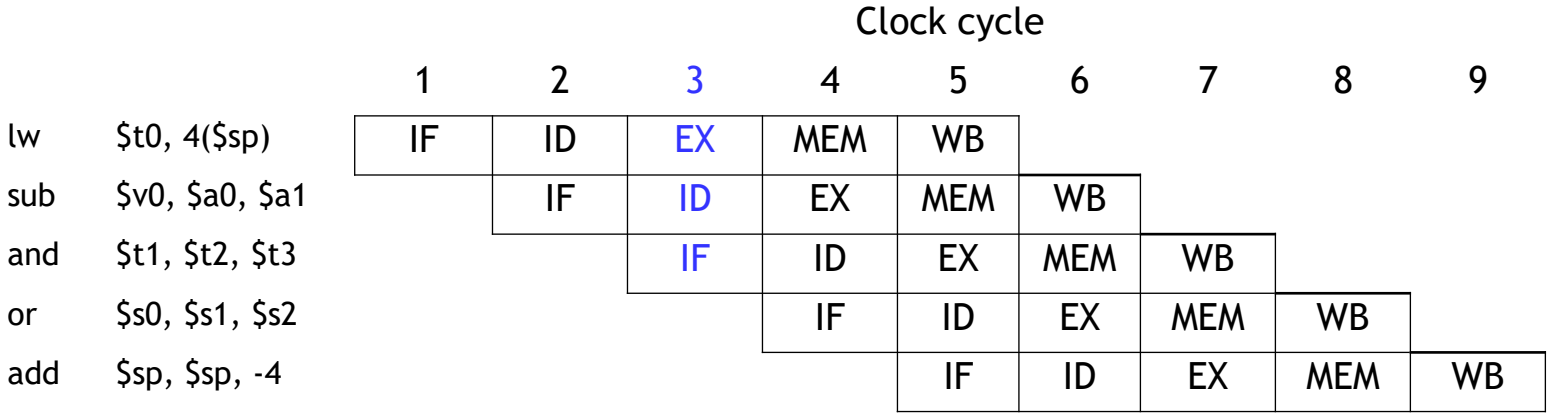
Five Stages of an Instruction



IF: Instruction Fetch, Fetch the instruction from the Instruction Memory
ID: Registers Fetch and Instruction Decode
EX: Calculate the memory address
MEM: Read the data from the Data Memory
WB: Write the data back to the register file

Five Stages of an Instruction

- Pipelining is an implementation technique in which multiple instructions are overlapped in execution.



Why memory segmentation?

- All the Internal registers of 8086 are only 16 bits wide

How do you use 16-bit registers to control 20-bit memory?

Why memory segmentation?

Limited Addressing Capability

- With 8086 using only 16 bit register for storing address, allows it to directly address up to 2^{16}=65,536 bytes (64 KB) of memory in a flat memory model.

- However, the 8086 was designed to work with **1 MB of memory**. This is achieved through segmentation.

Why memory segmentation?

BITS Pilani
K K Birla Goa Campus

Efficient Memory Utilization

- Segmentation divides memory into logically related sections, such as:
  - **Code Segment (CS):** Stores executable code.
  - **Data Segment (DS):** Stores variables and data.
  - **Stack Segment (SS):** Stores the stack for subroutine calls and interrupts.
  - **Extra Segment (ES):** Used for additional data or special operations.
- This logical division helps in organizing and managing memory efficiently.

Why memory segmentation?

Relocatable Code

- By changing the segment base address, programs can run in different memory locations without modifying the actual code.

Modular Programming

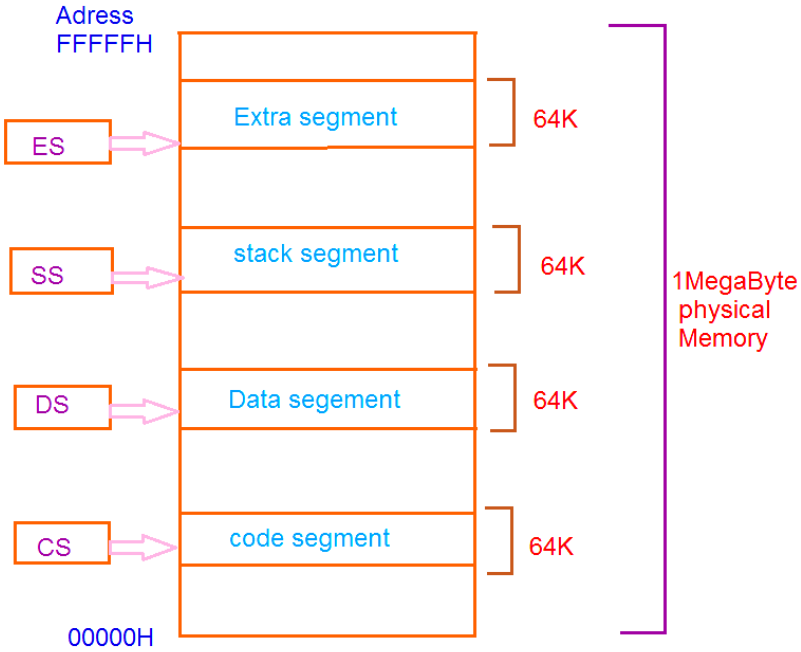
- Segments allow separation of different parts of a program (e.g., code, data, stack), which is useful for structured and modular programming.

Why memory segmentation?

Efficient Use of Stack

- Stack operations are confined to the **stack segment**, ensuring that stack-related data does not overwrite code or data segments.

Memory Segmentation



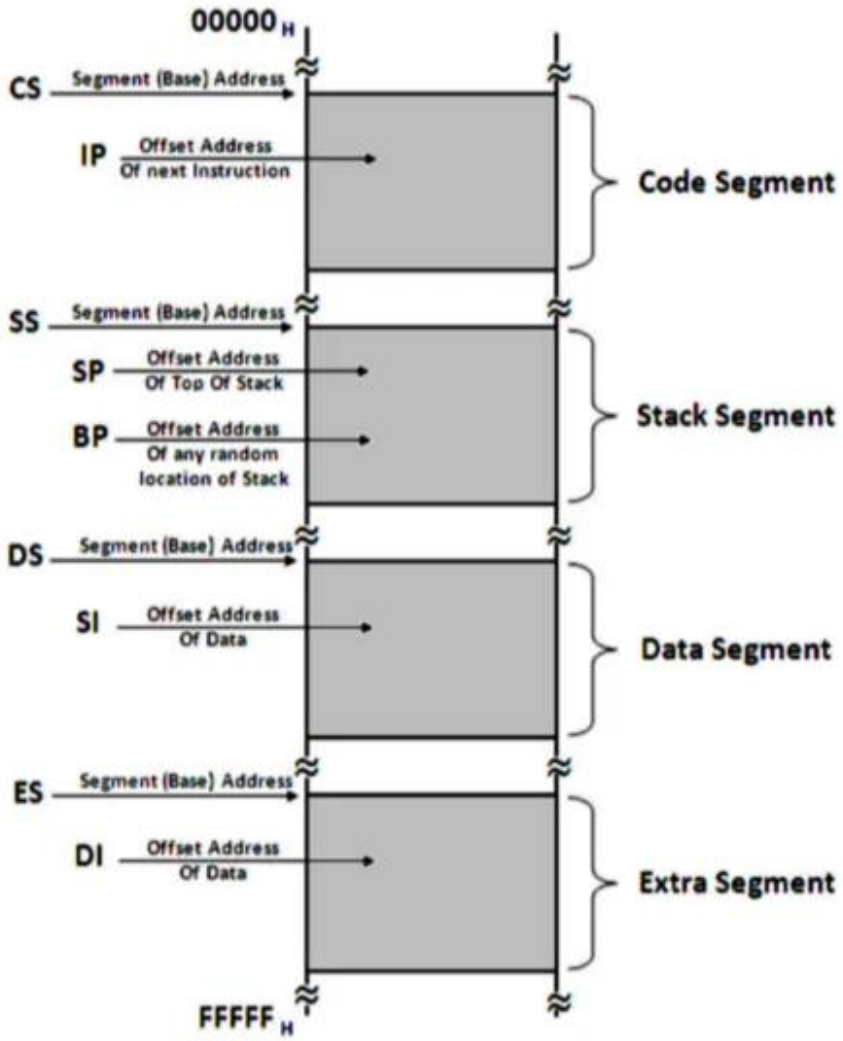
Memory Segmentation

PHYSICAL ADDRESS = SEGMENT ADDRESS X 10H + OFFSET ADDRESS

Physical address = 1000 X 10 + 2345

Physical address = 12345H

This allows the 8086 to address 16 × 64 KB=1 MB.



Example

- If **CS = 1234H** and **IP = 5678H**, the physical address of the instruction being executed is:

Example

- If **CS = 1234H** and **IP = 5678H**, the physical address of the instruction being executed is:

Physical Address = (CS × 10H) + IP
= (1234H × 10H) + 5678H
= 12340H + 5678H
= 179B8H

C Vs 8086

```c
#include <stdio.h>

int main() {
    int num1, num2, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &num1, &num2);
    sum = num1 + num2;
    printf("Sum = %d", sum);
    return 0;
}
```

```
; Input num1
mov ah, 01h
int 21h
sub al, 30h          ; Convert ASCII to integer
mov bl, al           ; Store in num1

; Input num2
mov ah, 01h
int 21h
sub al, 30h          ; Convert ASCII to integer
mov bh, al           ; Store in num2
```

```
; Add num1 and num2
mov al, bl           ; Load num1 into AL
add al, bh           ; Add num2 to AL
mov sum, al          ; Store the result in sum

; Display result
mov ah, 09h
lea dx, result
int 21h
```

MOV Instruction

- The MOV instruction copies data from a source operand to a destination operand without altering the source.

MOV destination, source

• **destination:** The operand where the data will be stored.
• **source:** The operand containing the data to be moved.

MOV AX, BX ; Copies the contents of BX into AX

ADD Instruction

- The ADD instruction performs binary addition of two operands.

ADD destination, source

- **destination:** The operand where the result will be stored.
- **source:** The operand containing the value to be added.

```
ADD AX, BX  ; AX = AX + BX

MOV AX, 1234H    ; Load AX with 1234H
MOV BX, 5678H    ; Load BX with 5678H
ADD AX, BX       ; AX = AX + BX (AX = 1234H + 5678H)
```

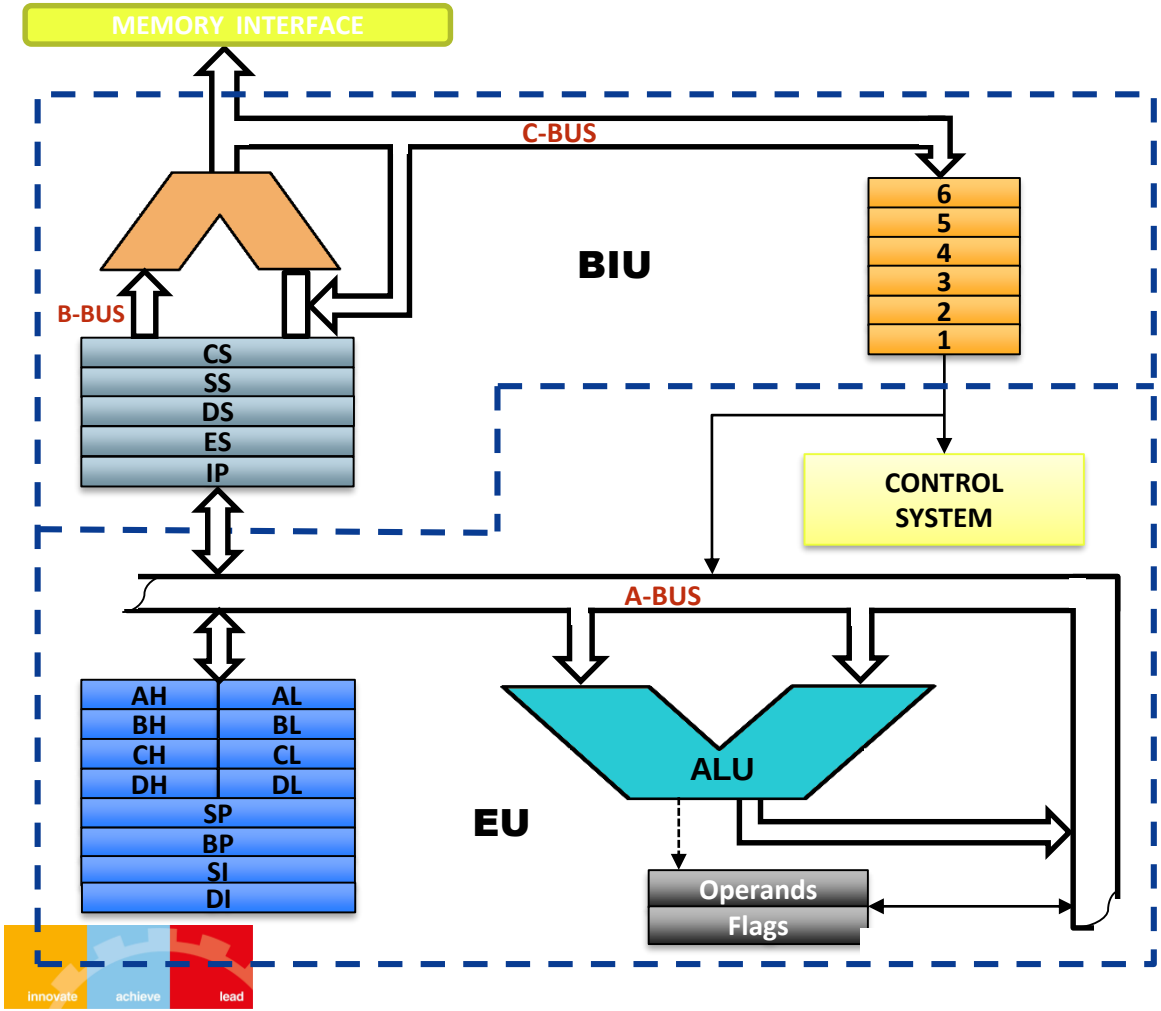
Intel 8085 vs 8086

	Intel 8085	Intel 8086
Data Bus Width	**8-bit** (handles 1 byte at a time)	**16-bit** (handles 2 bytes at a time)
Address Bus	16-bit (64 KB Memory)	20 bit (1 MB Memory)
Pipelining	No (Fetch-Execute-Repeat)	Yes (Fetch While Executing)
Memory Org	Single Flat Memory Space	Segmented (Code, Data , Stack)
Modes	Single Mode Only	Two modes (Min and Max)

The 8086 Microprocessor

- First 16-bit microprocessor released in 1978
- 20-bit address bus, 1,048,576 memory locations
- 16-bit data bus, read or write 16 bits or 8 bits at a time
- Segmentation of memory for increasing execution speed
- Early implementation of pipelining

Block Diagram of 8086



- 8086 CPU is divided into two parts:
  - Bus Interface Unit (BIU)
  - Execution Unit (EU)
- BIU handles all transfers of data and addresses on the buses for EU
- EU tells BIU where to fetch instructions or data from, decodes and executes instruction.

BITS Pilani
K K Birla Goa Campus

THANK YOU